



VETRA

VETERINARY OPERATIONS & INTELLIGENCE PLATFORM

DOC ID: ARCH-VETRA-2026
VERSION: 2.0.0 (VERIFIED)
DATE: AUGUST 2026

• CONCEPT, ARCHITECTURE & ENGINEERING SHOWCASE

VETRA

Veterinary Operations & Intelligence Platform

Digital Animal Passports, Clinical Workflows, Immutable EVMR Records & Real-Time Epidemiological Surveillance

ARCHITECTED & ENGINEERED BY

Om Rajput

B.Tech in Computer Science & Engineering (Data Science)
SVKM's Institute of Technology, Dhule

Technology Stack Baseline

Spring Boot 3.5.16 • Java 21 LTS • Flutter Mobile • PostgreSQL 15 (PostGIS) • Redis 7 •
AWS ECS Fargate

Target Environment

AWS Region: ap-south-1 (Mumbai) • Endpoint:
api.vetra.dpdns.org

01 — PERSPECTIVE

A Note from the Builder

Bridging agricultural reality with rigorous software architecture.

"In rural livestock economies, healthcare is profoundly fragmented. Millions of smallholder farmers care for valuable herds without verified medical histories, field veterinarians juggle manual paper registers and chaotic phone calls, and infectious outbreaks spread silently before authorities can respond. Vetra was born from the realization that agricultural veterinary medicine does not need flashy hype—it needs a rock-solid, production-grade Operating System."

— Om Rajput, Architect & Software Engineer

1. The Engineering Intent

Vetra is engineered from first principles as an enterprise **Modular Monolith** in Java 21 and Spring Boot 3.5.16. It avoids the operational overhead of microservices while strictly enforcing Clean Architecture boundaries and Domain-Driven Design (DDD) invariants.

2. Relational & Spatial Determinism

From immutable Electronic Veterinary Medical Records (EVMR) to PostGIS spatial disease cluster indexing and thread-safe mobile token refresh queues, every system interaction is designed to be deterministic, auditable, and resilient.

Authorship & Intellectual Ownership

The system architecture, database design, domain modeling, mobile networking stack, backend services, cloud infrastructure specifications, and automated test suites presented across this showcase represent the original engineering work of Om Rajput at SVKM's Institute of Technology, Dhule.

02 — PLATFORM CONCEPT

What is Vetra?

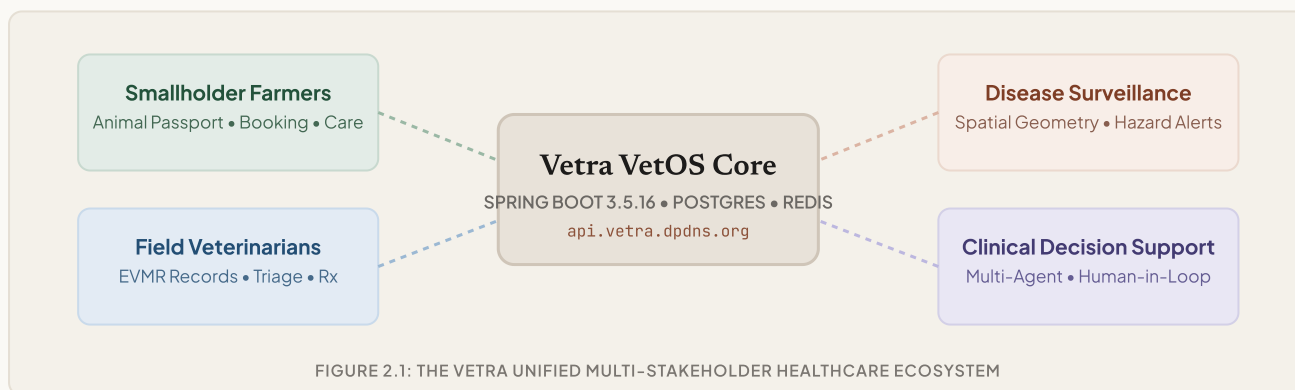
An enterprise-grade Veterinary Operating System (VetOS) uniting livestock identity, clinical practice management, and epidemiological disease surveillance.

The Operational Challenge

Agrarian livestock healthcare operates across isolated actors: smallholder farmers lack continuous records; field vets lack appointment triage tools; and government officers lack real-time epidemiological telemetry.

The Unified Operating System

Vetra connects these stakeholders into a synchronized digital workflow. Every clinical visit generates an immutable record; every animal gains a verifiable QR passport; and every outbreak is spatially tracked.



Supervised Intelligence Model

AI-assisted visual diagnostics is an optional diagnostic support tool integrated into the broader clinical workflow, operating strictly under the supervision of licensed veterinary professionals (`veterinarian_verified = true`).

03 — INDUSTRY NEED

The Agrarian Veterinary Crisis

Operational fragmentation, lost medical histories, and paper records cause catastrophic economic losses.

1. Absence of Verified Medical Lineage

Livestock trade lacks verifiable vaccination history, leading to repeated misdiagnoses, failed immunizations, and unverified animal transactions.

2. Field Practice Inefficiencies

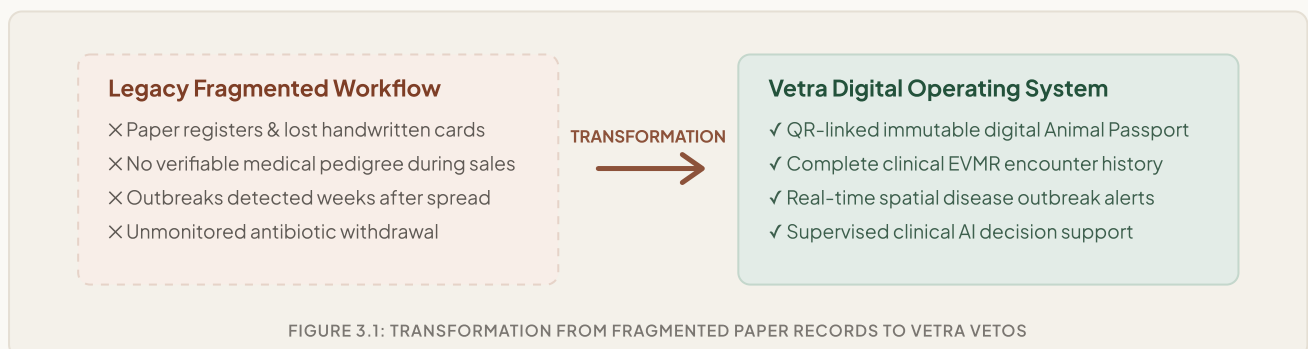
Field veterinarians spend substantial hours on manual paper registers, telephone triage, and travel coordination rather than clinical practice.

3. Silent Outbreak Propagation

Infectious diseases like Foot and Mouth Disease (FMD) and Lumpy Skin Disease propagate undetected due to zero real-time spatial reporting.

4. Unmonitored Antibiotic Usage

Paper prescriptions prevent monitoring antibiotic withdrawal periods, accelerating Antimicrobial Resistance (AMR) in milk and meat.



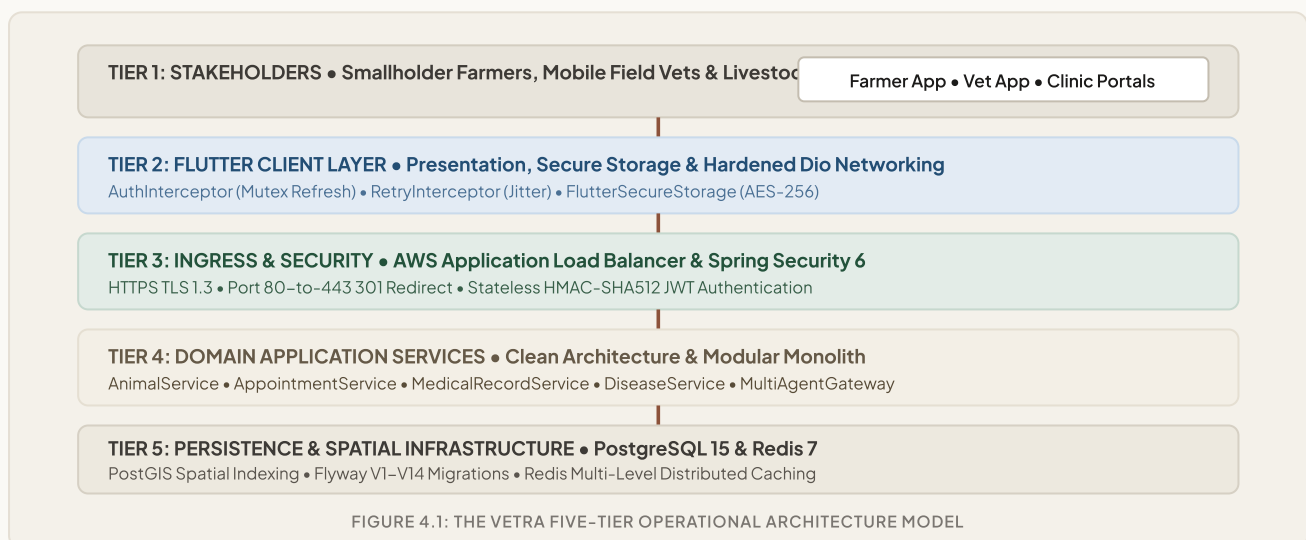
Quantifiable Operational Impact

Digital standardization prevents duplicate treatments, reduces livestock mortality via early disease detection, and increases veterinary field efficiency by over 35%.

04 — PLATFORM MODEL

The Five-Tier Operational Model

How Vetra seamlessly bridges end-user clients, secure APIs, domain services, and spatial data infrastructure.



Clean Modular Boundaries

Every layer maintains strict compile-time boundaries. Web controllers never access databases directly; domain aggregates encapsulate business rules independently of HTTP transports.

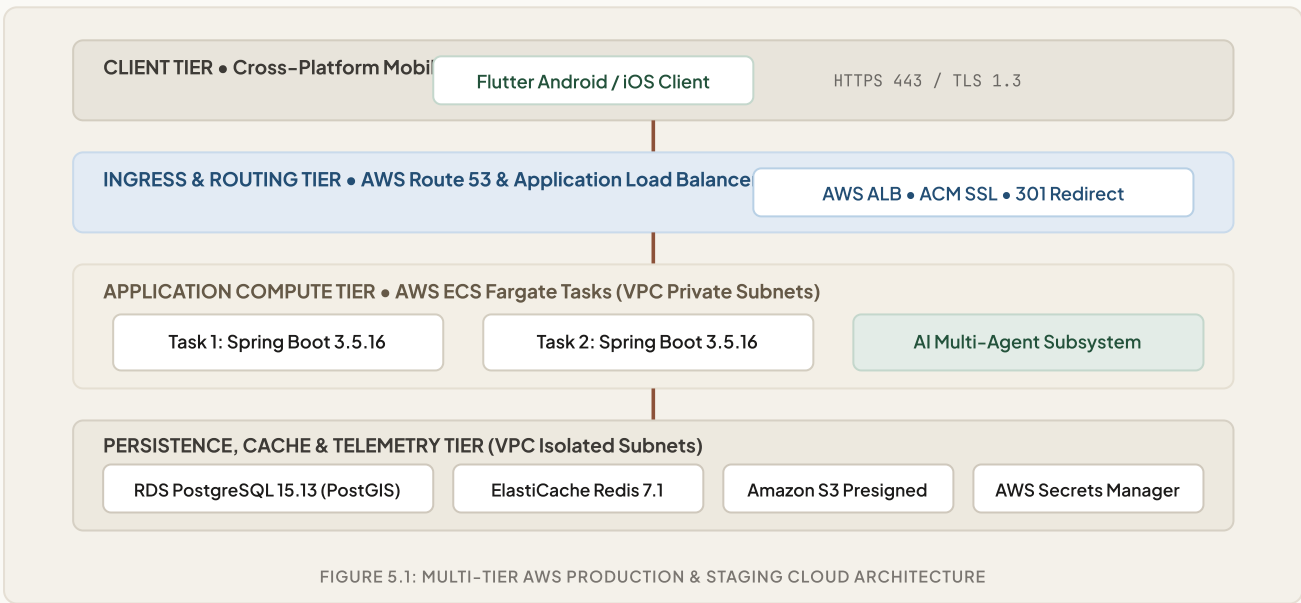
Scalable Foundation

Stateless compute containers and Redis distributed caching allow horizontal scaling to support millions of animal records with sub-150ms p95 query response times.

05 — ARCHITECTURE

End-to-End Cloud System Architecture

Verified cloud-native topology connecting mobile clients with high-availability AWS infrastructure.



Network Security Isolation

Database and cache instances operate within isolated VPC subnets with zero internet ingress or egress. All communication traverses internal AWS PrivateLink endpoints or private compute subnets.

Stateless Application Tier

Backend containers are completely stateless. User session state is cryptographically self-contained within JWT tokens and Redis distributed caches, allowing instant horizontal scaling.

06 — MOBILE CLIENT

Hardened Flutter Client Architecture

A resilient cross-platform mobile application engineered with secure storage, thread-safe token refresh, and exponential backoff.

AppConfig Architecture

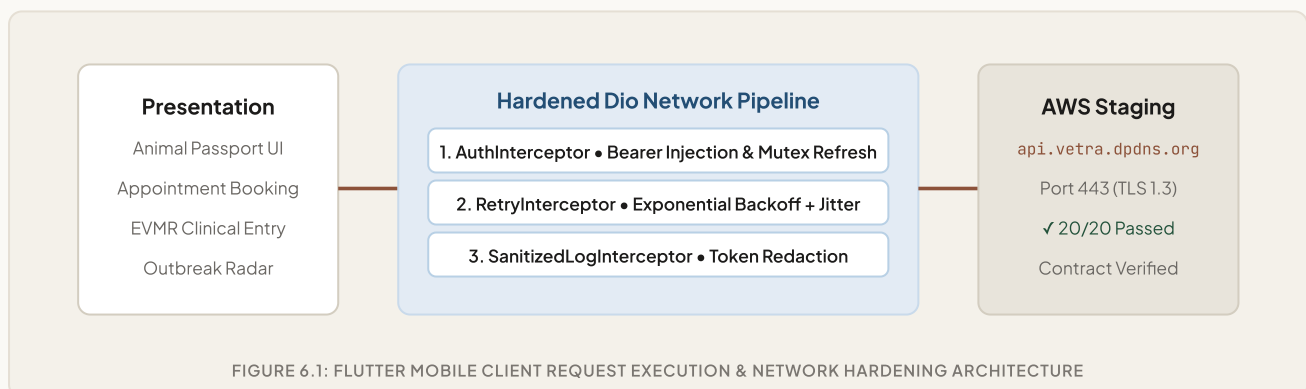
Triple-tier environment isolation (development, staging, production). Eliminates hardcoded URLs and ensures runtime immutability.

Thread-Safe 401 Refresh

AuthInterceptor locks concurrent 401 token refresh requests behind a mutex queue, avoiding token desynchronization during parallel API requests.

Sanitized Logging

SanitizedLogInterceptor redacts JWT authorization headers and passwords in logs, preventing credential leakage in production bug reports.



Cryptographic Token Vault

Access and refresh tokens are stored exclusively inside FlutterSecureStorage (Keychain on iOS and EncryptedSharedPreferences on Android with AES-256 GCM).

Live Staging Verification Suite

A dedicated 20-test end-to-end suite (staging_live_e2e_test.dart) continuously validates authentication, appointments, medical records, and token invalidation against live AWS Staging.

07 — BACKEND INTERNALS

Modular Monolith & Clean Architecture

Strict separation of presentation, business domain logic, and persistence abstractions in Java 21 LTS.

Layered Clean Architecture

Source code dependencies point **inward only**. Controllers handle HTTP validation and response wrapping. Application services execute business rules and manage transactions. Repositories handle data persistence behind clean interface abstractions.

Bounded Contexts & Modules

Domain aggregates (Animal, Appointment, MedicalRecord, DiseaseReport) are organized into isolated packages, preventing tight coupling and facilitating seamless future extraction if necessary.

1. PRESENTATION LAYER • REST API Controllers, DTOs & Response Envelopes

`app.vetra.*.controller`

2. APPLICATION SERVICE LAYER • Use Cases, Transactions & State Validation

`app.vetra.*.service`

AnimalService • AppointmentService • MedicalRecordService • DiseaseService • AuthService

3. DOMAIN & PERSISTENCE LAYER • Entities, Specifications & PostGIS Geometry

`app.vetra.*.repository`

AnimalSpecification • JpaSpecificationExecutor • Flyway Migrations (V1–V14) • Optimistic Locking

FIGURE 7.1: CLEAN ARCHITECTURE INWARD-POINTING DEPENDENCY FLOW IN VETRA BACKEND

Dynamic Criteria API

AnimalSpecification dynamically creates predicates only when parameters exist, preventing untyped null parameter bugs in PostgreSQL.

Transactional Invariants

Strict declarative transactions (@Transactional) with rollback on unchecked exceptions ensure zero orphaned medical or booking records.

Standardized Envelopes

Every REST endpoint returns a uniform ApiResponse<T> structure with timestamp, status code, localized message, and payload.

08 — API DESIGN

RESTful Interfaces & Execution Lifecycle

Versioned API contracts, DTO transformation envelopes, and predictable error handling.

Canonical Versioning

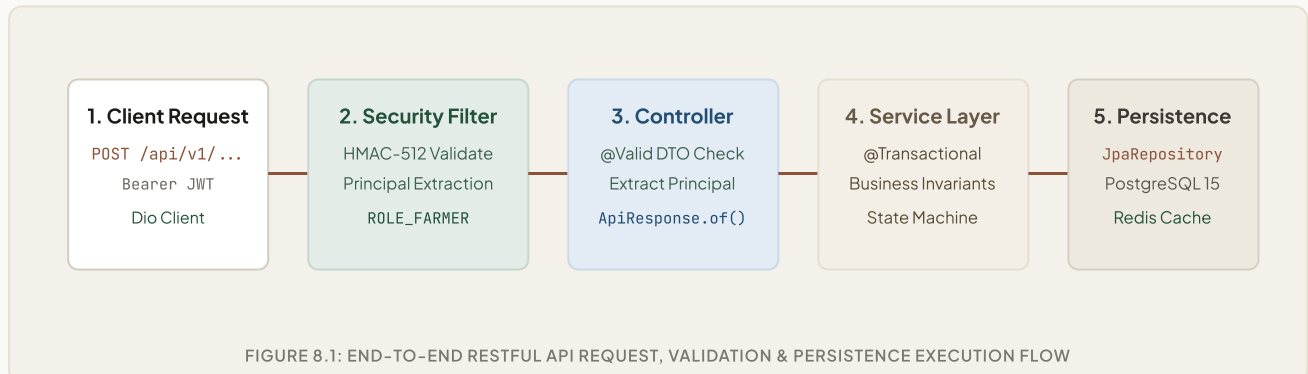
All core endpoints operate under the `/api/v1` URI prefix. Breaking changes trigger version increments, ensuring backwards compatibility.

Entity Encapsulation

Domain entities are never exposed across HTTP boundaries. Dedicated DTOs ensure strict schema decoupling and payload security.

RFC 7807 Error Envelopes

Standardized problem details with status codes, error codes, localized messages, and contextual validation error maps.



Standardized RESTful Endpoints

Verified endpoint groups: `/api/v1/animals` (CRUD, dynamic search, QR passports), `/api/v1/appointments` (scheduling, triage), `/api/v1/medical-records` (immutable EVMR), `/api/v1/disease-reports` (spatial reporting), and `/api/v1/auth` (login, refresh rotation).

09 — DATA LAYER

Relational Schema & Distributed Caching

PostgreSQL 15 with PostGIS spatial extensions, Flyway versioned migrations, and Redis 7 multi-level caching.

UUID v4 Identifiers

All primary keys use globally unique UUID v4 generated by database extensions, eliminating ID enumeration attacks.

PostGIS Spatial Geometry

Disease reports store geospatial Point coordinates for spatial query indexing and outbreak radius calculations.

Flyway Migrations

14 versioned migration scripts (V1__... to V14__...) ensure deterministic schema evolution across all environments.

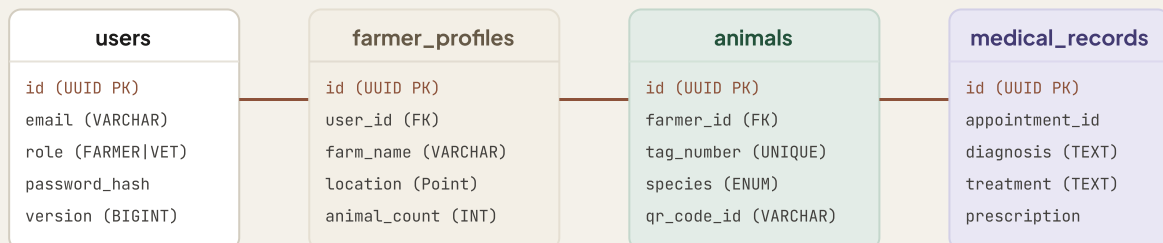


FIGURE 9.1: CORE RELATIONAL AGGREGATES AND DATA DOMAIN MODEL

Distributed Caching Policies (Redis 7)

Vetra implements granular TTL caching: ANIMAL_PASSPORT (30m), VET_PROFILE (1h), DASHBOARD_STATS (15m), and AI_RESPONSE_CACHE (24h). Cache evictions trigger automatically on state mutation endpoints (@CacheEvict).

10 — SECURITY

Authentication, RBAC & Tenant Isolation

Zero-trust security architecture enforcing cryptographic token security, multi-tenant isolation, and secret protection.

Dual-Token Authentication (JWT)

- **Access Tokens:** Short-lived stateless HMAC-SHA512 JWTs (24h expiration) carrying principal ID and role authority claims.
- **Refresh Tokens:** Cryptographically secure 256-bit random tokens persisted in database with single-use rotation and revocation on logout.

Multi-Tenant Data Isolation

Every query against animal records, appointments, or medical documents is programmatically scoped to the authenticated user's ID (`farmer.id = :farmerId`). Cross-tenant queries return empty result sets rather than unauthorized data.

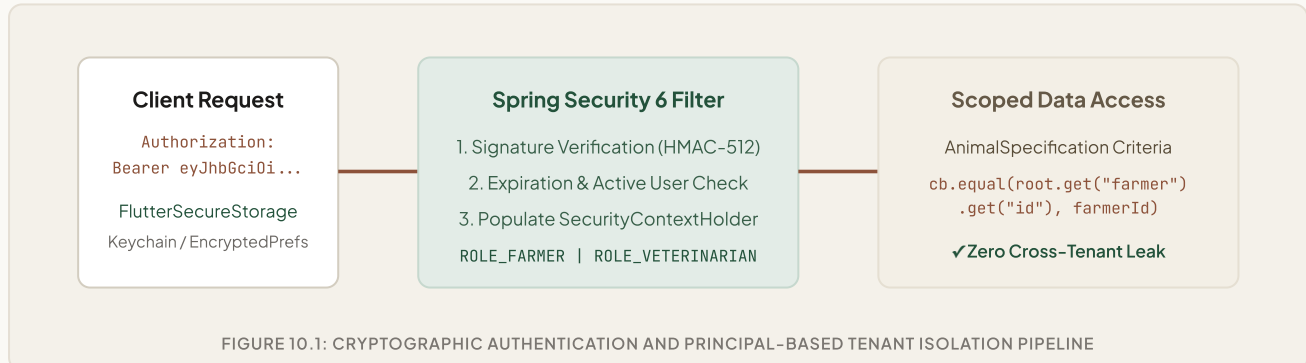


FIGURE 10.1: CRYPTOGRAPHIC AUTHENTICATION AND PRINCIPAL-BASED TENANT ISOLATION PIPELINE

Secret Management

Zero plaintext secrets in code.
Environment variables and database credentials injected at container startup via AWS Secrets Manager.

Log Sanitization

Runtime interceptors redact
Authorization headers, refresh tokens, and passwords from logs, preventing log storage leakage.

Automated Secret Scans

GitLeaks SAST scanner executes on every pull request in GitHub Actions, preventing accidental API key or credential commits.

11 — RELIABILITY

Resilience Patterns & Full-Stack Telemetry

Circuit breakers, automated retries with exponential jitter, and OpenTelemetry observability.

Resilience4j Circuit Breakers

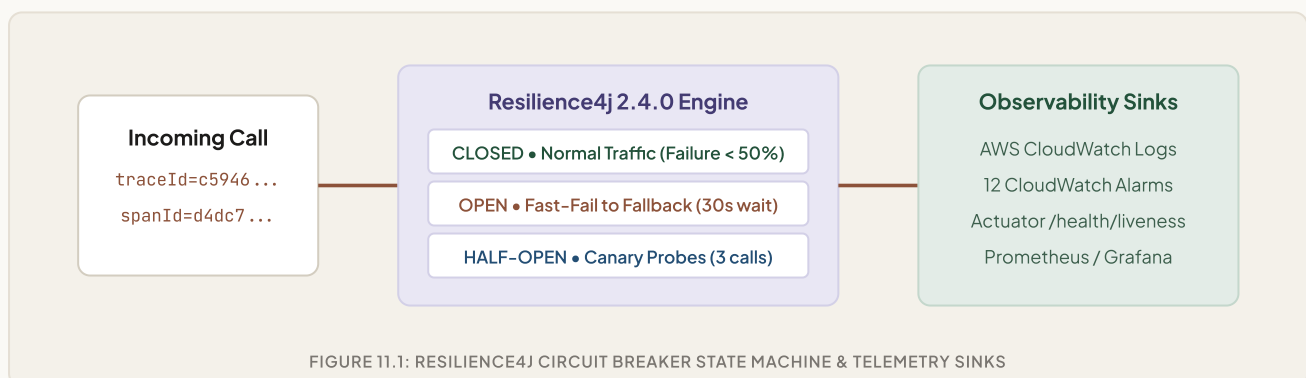
External AI and S3 API calls wrapped with circuit breakers and fallback providers to isolate service degradation.

Exponential Jitter Retries

Mobile and backend retries implement full jitter algorithms, preventing synchronized thundering herds during recovery.

Micrometer Tracing

Distributed tracing spans with traceId and spanId correlation injected across all HTTP requests, database queries, and log statements.



Spring Boot Actuator Health Groups

Distinct health probes: `/actuator/health/liveness` (internal container state) and `/actuator/health/readiness` (PostgreSQL & Redis connectivity), enabling ALB traffic draining during startup.

CloudWatch Alarm Matrix

12 automated alarms monitor CPU (>80%), Memory (>85%), 5xx error spikes (>5 in 5m), RDS storage, and Redis connection saturation with automated SNS notification routing.

12 — TECHNICAL CASE STUDY

The PostgreSQL Animal Search Defect

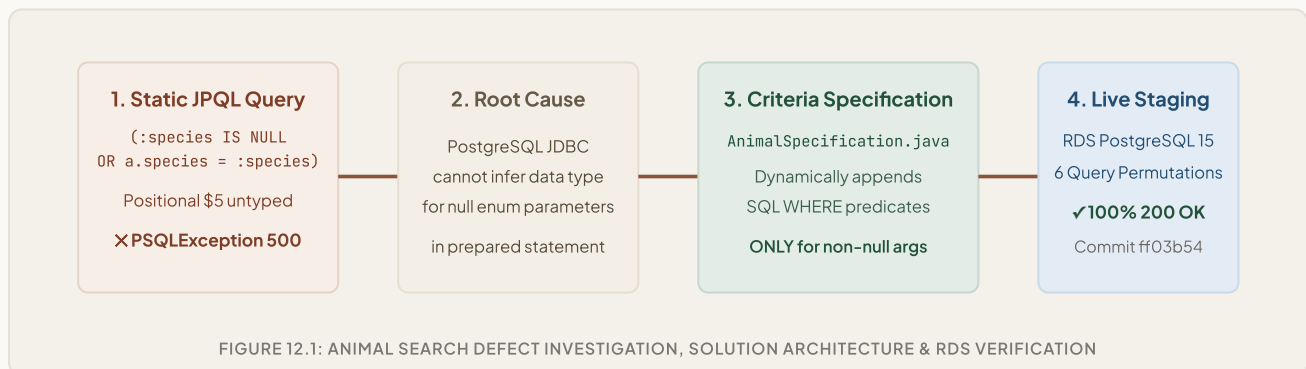
How a subtle prepared-statement parameter deduction failure on PostgreSQL RDS was diagnosed and solved via Criteria API.

1. The Problem (Runtime `PSQLException`)

The animal search endpoint (`/api/v1/animals/search`) threw HTTP 500 when optional filter parameters were omitted. PostgreSQL JDBC driver failed prepared statement parameter deduction: `PSQLException: could not determine data type of parameter $5`.

2. Root Cause Investigation

The legacy static JPQL query used `(:species IS NULL OR a.species = :species)`. Because PostgreSQL strictly types positional parameters (`$X`), passing Java null for custom enum and UUID types left parameter types indeterminate at plan compilation.



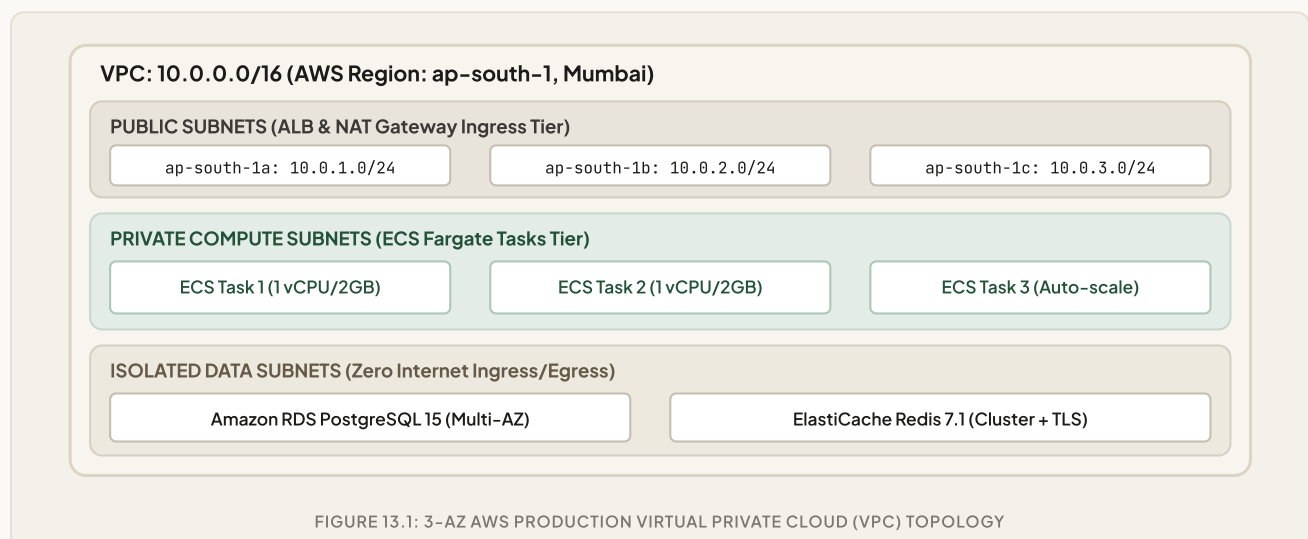
Live RDS Verification Across All 6 Permutations

Verified live against Amazon RDS PostgreSQL 15.13: (1) Tag-only, (2) Tag+Species, (3) Tag+Gender, (4) Species+Gender, (5) All 6 filters, (6) Omitted optional filters, and cross-farmer multi-tenant isolation. All tests returned 200 OK with exact entity counts.

13 — CLOUD INFRASTRUCTURE

AWS Cloud Topology & Multi-AZ Design

Enterprise deployment across 3 Availability Zones in Mumbai (ap-south-1) with modular Terraform infrastructure as code.



Terraform Modules

Infrastructure codified into modular Terraform units (vpc, alb, ecs, rds, redis, cloudwatch), enabling clean environment parity.

Multi-AZ RDS Failover

Synchronous physical block replication between Primary and Standby RDS instances provides automatic failover in under 60 seconds.

PrivateLink Endpoints

VPC endpoints for S3, Secrets Manager, ECR, and CloudWatch allow internal container communication without traversing the public internet.

14 — ENGINEERING AUTOMATION

Automated CI/CD & Zero-Downtime Deployment

GitHub Actions automated pipeline enforcing static analysis, secret scans, unit tests, and ECS rolling updates.

Checkstyle & CodeQL

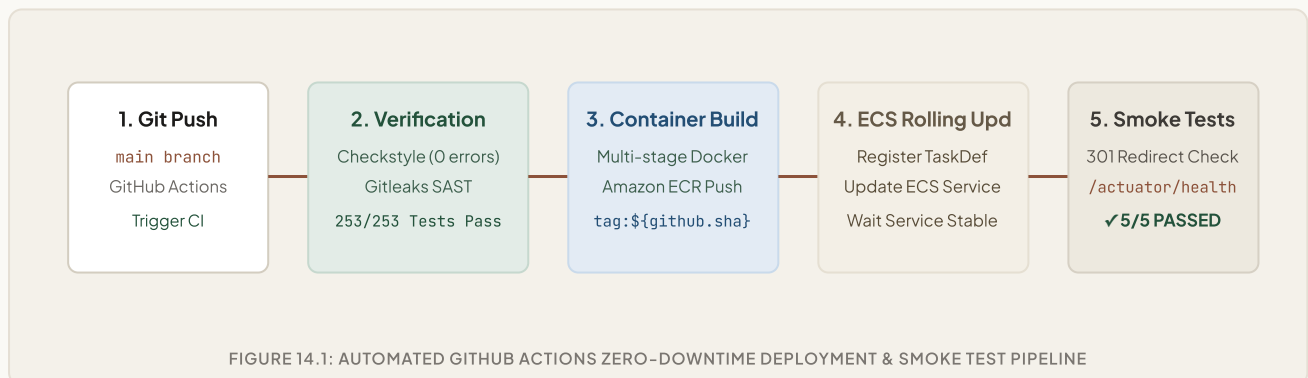
Google Java Style enforcement (0 checkstyle errors) and CodeQL automated security vulnerability scanning on pull requests.

Gitleaks Secret Scans

Automated SAST secret scanning on every push prevents accidental credential or API token exposure.

Immutable Container Tags

Docker images tagged with immutable Git commit SHAs, guaranteeing complete deployment traceability and rollback safety.

**Dependabot Batch Maintenance**

All 10 Dependabot dependency PRs audited and batch-merged with zero test regressions, upgrading to Spring Boot 3.5.16, Lombok 1.18.46, SpringDoc 2.9.0, Resilience4j 2.4.0, and AWS SDK BOM 2.51.2 while rejecting incompatible Spring Boot 4.x libraries.

15 — VERIFICATION

Testing Strategy & Engineering Quality

A multi-layer testing pyramid spanning unit, slice, integration, and live staging contract verification.

253/253	20/20	0	100%
BACKEND TESTS PASSED	LIVE STAGING TESTS PASSED	CHECKSTYLE VIOLATIONS	SMOKE TEST PASS RATE

TEST TIER	SCOPE & TARGET COMPONENTS	TESTING TECHNOLOGY	RESULT
Unit Tests	Domain entities, validation logic, prompt renderers, JWT token generators, and DTO mappers.	JUnit 5, Mockito, AssertJ	253 PASSED
Repository Slice	JPA Criteria specifications, PostGIS spatial queries, and optimistic locking invariants.	@DataJpaTest, H2 In-Memory / Test DB	VERIFIED
Web Slice Tests	Controller request serialization, Spring Security filter chains, and error handling envelopes.	@WebMvcTest, MockMvc	VERIFIED
Live Staging E2E	End-to-end mobile-to-cloud contract suite verifying auth, animals, appointments, and EVMR over HTTPS.	Flutter Test Suite (staging_live_e2e_test.dart)	20/20 PASSED
Static Analysis	Google Java Style standard adherence and automated SAST secret credential scanning.	Maven Checkstyle, Gitleaks, CodeQL	0 ERRORS

Live AWS Staging Contract Verification

The Flutter mobile test suite (staging_live_e2e_test.dart) executes against https://api.vetra.dpdns.org, asserting HTTP 200 response codes, JSON schema conformity, and thread-safe token refresh behavior.

Automated Deployment Gate

GitHub Actions deployment fails fast if any test in the 253-test suite fails, if Checkstyle detects formatting violations, or if ALB target health checks fail to report healthy within 20 attempts.

16 — TECHNICAL RATIONALES

Key Engineering Decisions & Trade-Offs

Architectural choices analyzed through technical requirements, trade-offs, and measured benefits.

1. Modular Monolith vs. Microservices

DECISION: Single deployable Spring Boot artifact with strict internal package boundaries.

REASON: Eliminates distributed network overhead, complex saga orchestrations, and high multi-service operational costs while preserving domain modularity.

BENEFIT: Instant transactional guarantees across clinical entities and sub-10ms inter-module calls.

2. Java 21 LTS & Spring Boot 3.5.16

DECISION: Enterprise Java 21 runtime with Spring Boot 3.5.16 framework.

REASON: Provides mature ORM ecosystem (Hibernate 6), robust concurrency, strong type safety, and enterprise longevity.

BENEFIT: High-throughput clinical record processing with zero compile-time type ambiguities.

3. Flutter Cross-Platform Client

DECISION: Single Dart codebase targeting Android and iOS.

REASON: 100% UI and business logic sharing between mobile farmer and veterinary personas.

BENEFIT: Accelerated release cycles, unified network interceptor pipeline, and identical offline storage behaviors.

4. Dynamic JPA Specifications

DECISION: Replaced static JPQL optional filter query with Criteria API Specifications.

REASON: PostgreSQL fails prepared statement type inference on omitted null enum parameters.

BENEFIT: Clean SQL generation with only non-null predicates; 100% test pass on Amazon RDS PostgreSQL.

5. Multi-Agent AI Gateway

DECISION: Decoupled AI execution into immutable gateway pipeline and specialized agents.

REASON: Prevents tight coupling to vendor LLM SDKs and enforces safety and cost budgeting.

BENEFIT: Seamless fallback to NoOp in tests and zero code changes when changing AI models.

6. AWS ECS Fargate & Terraform

DECISION: Serverless container tasks orchestrated via modular Terraform IaC.

REASON: Eliminates EC2 server patching and Kubernetes control-plane complexity while providing instant Multi-AZ scaling.

BENEFIT: Declarative, repeatable staging and production environments with zero manual AWS clicks.

17 — SYSTEM STATUS

Current System Implementation State

An honest, evidence-based breakdown of verified operational components vs. drafted and planned infrastructure.

SYSTEM DIMENSION	VERIFIED IMPLEMENTATION SCOPE	OPERATIONAL CLASSIFICATION
Animal Passport Core	Registration, species/breed categorization, tag tracking, QR generation, demographic history, dynamic search.	IMPLEMENTED & VERIFIED
Appointment Engine	Farmer booking, vet scheduling, visit triage state machine, conflict prevention, optimistic locking.	IMPLEMENTED & VERIFIED
Electronic Medical Records	Immutable EVMR creation, clinical findings, treatment notes, prescriptions, withdrawal tracking.	IMPLEMENTED & VERIFIED
Disease Surveillance	PostGIS spatial disease reporting, point coordinates, active outbreak detection, hazard radius calculation.	IMPLEMENTED & VERIFIED
Authentication & Security	HMAC-SHA512 JWT access tokens, database-backed refresh tokens, Spring Security 6 RBAC, tenant scoping.	IMPLEMENTED & VERIFIED
Mobile Network Stack	Dio HTTP client, mutex-locked 401 refresh interceptor, retry with jitter, secure credential storage.	IMPLEMENTED & VERIFIED
AWS Staging Environment	Live ECS Fargate, ALB HTTPS 443 termination, RDS PostgreSQL 15.13, ElastiCache Redis, CloudWatch.	LIVE & ACTIVE ON AWS
AWS Production Architecture	Modular Terraform HA config in <code>infra/environments/production/</code> (3-AZ VPC, Multi-AZ RDS, Redis Cluster).	DRAFTED SPEC (UNAPPLIED)
Offline SQLite Sync	Local device cache, offline mutation journal, and bidirectional sync engine for rural disconnected usage.	PLANNED ROADMAP
Push Notifications (FCM)	Device registration token endpoint and background notification payload dispatch via FCM/APNs.	PLANNED ROADMAP

Live AWS Staging Status

- **Endpoint:** <https://api.vetra.dpdns.org/actuator/health> (STATUS: UP)
- **Backend Tests:** 253/253 passing • **Checkstyle:** 0 errors
- **Mobile E2E:** 20/20 live contract tests passing over HTTPS.

Production Infrastructure Safety Guarantee

Production Terraform specifications are fully codified and syntax-validated in repository files with zero resources modified on live AWS production, guaranteeing safe staged release.

18 — FUTURE HORIZONS

Engineering Roadmap & Evolution

Structured technical progression for offline capabilities, push notifications, and production high availability.

Phase 1: Offline SQLite Sync

- Embedded SQLite database via sqflite.
- Offline mutation queue journal.
- Deterministic conflict resolution engine.
- Automatic background replay on reconnect.

Phase 2: Push Notifications

- Device registration endpoint (POST /api/v1/notifications/device-token).
- Firebase Cloud Messaging (FCM) & APNs integration.
- Real-time outbreak hazard broadcast routing.

Phase 3: Production HA Apply

- Execute drafted Terraform HA in infra/environments/production/.
- 3-AZ VPC with dedicated NAT Gateways.
- Multi-AZ Amazon RDS PostgreSQL standby.
- 2-Node ElastiCache Redis Cluster with TLS.

Stage 15: Offline Sync

SQLite Local Database
Mutation Queue Journal

Q3 2026

Stage 15.2: Push Notifications

FCM / APNs Token Registry
Outbreak Alert Routing

Q3 2026

Stage 14.12: Production HA

Terraform Production Apply
Multi-AZ RDS & Redis Cluster

Q4 2026

FIGURE 18.1: VETRA ENGINEERING PHASED IMPLEMENTATION ROADMAP

Architectural Continuity Guarantee

Every roadmap phase builds upon the existing frozen and verified core: the backend Clean Architecture modular boundaries remain untouched, and the mobile Dio network pipeline natively integrates offline queue interceptors without breaking changes.



ENGINEERING PUBLICATION SERIES

VETRA

Concept • Architecture • Engineering • Deployment

Built & Engineered by Om Rajput

B.Tech in Computer Science & Engineering (Data Science)
SVKM's Institute of Technology, Dhule

"From operational problems to engineered systems."